Iris Hsieh

Portfolio Case Studies

Senior Product Designer
Last updated: May 2026

Contents

01. Designing with AI — Wren AI · 2026

02. One Product, Three Minds — Wren AI · 2026

03. Embedded Threads — Wren AI · 2026

04. Closing the Loop — Wren AI · 2026

05. Notification Framework — Google Pixel · 2025

06. Rezio Design System Overhaul — KKday Rezio · 2024–25

07. SaaS Onboarding Revamp — KKday Rezio · 2024–25

08. Bicycle Lane Inspection — LSBG Hamburg · 2024

09. A41 — All For One — None Capital · 2022–23

10. Trip Planner — Claude Code Skill · 2026

11. Reflections on Design — Essay · 2026

About

AI · MOBILE · SAAS · DESIGN SYSTEM

Iris Hsieh, Co-prospering with AI, defining the next possibility of digital product experience.

Hi, I'm Ming Hsi Hsieh, also known as Iris Hsieh, a Senior Product Designer focused on AI, SaaS, mobile systems, design systems, and design workflow planning. Currently at Wren AI, previously at Google Pixel and KKday. I'm relocating to the UK, Europe, or Japan and open to opportunities across all three.

If you want to see how I design AI products, start with One Product, Three Minds, my rethink of conversational BI at Wren AI; Closing the Loop, on feedback systems that turn user signals into model instructions; or Bicycle Lane Inspection, an AI-assisted civic tool built for Hamburg's road maintenance team. For mobile systems work, there's the Notification Framework I led at Google Pixel, and A41, a B2C cross-store shopping experience I designed at Fable. For B2B SaaS, I built the Rezio Design System at KKday and redesigned the Rezio onboarding flow that cut vendor setup time by 38%.

For how I think about AI and design more broadly, browse my Playground, especially Designing with AI, where I document the workflow that lets one designer carry 6–8 FTE of scope across parallel projects. I'm currently building Trip Planner, a Claude Code skill that turns travel research into interactive HTML guides.

ralinzain@gmail.com LinkedIn CV ↓

Designing with AI

WREN AI · 2026

The premise

In early 2026, a designer at Anthropic declared that the traditional design process was dead. At the same time, I was a solo product designer at Wren AI, one person holding product design for a B2B AI platform, with 16 hours of my week swallowed by meetings and 3–4 projects running in parallel at any moment. This case study documents the 11-step AI workflow I built to make that actually possible.

My Role

Senior Product Designer at Wren AI. Also, by necessity: PM, researcher, UI/UX, tech writer, even motion designer. The workflow is how one person covers ~6–8 FTE of traditional scope without losing craft.

Timeline

2026 → Present

01. Why this workflow exists

The moment

Early 2026. An Anthropic designer publishes a post arguing that the traditional design process, discovery → research → wireframe → mockup → spec → handoff, is obsolete. Models can generate working interfaces faster than a designer can sketch one. The artifact (the Figma file) is no longer where the value lives.

I read it at a B2B AI startup where I was the only product designer. I didn't have the luxury of disagreeing. The reality was already pressing against me:

- A single designer covering product design, visual design, prototyping, research, and brand for an entire product.
- 16 hours/week structurally committed to meetings that couldn't be eliminated.
- A new release every week, each feature moves from brief to ship in under a week and a half of actual design time.
- Engineers who were already using Claude Code daily, if my handoffs weren't compatible with their workflow, I was the bottleneck.
- A startup budget that ruled out hiring.

The reframe

The old question, *"How do I get more time to design?"*, had no answer. The week was full. The new question became: *"How do I make every hour of mine count for five?"* That only worked if I stopped being the person making artifacts and started being the person directing a system that made them.

Claude Code became that system. Not as a tool I reach for occasionally, but as the central orchestration layer, the thing specs, prototypes, research summaries, marketing videos, Notion updates, and product docs all flow through. Below is the pipeline I ended up with.

02. The 11-step AI pipeline

Steps 00 → 11

Each step is a Claude Code skill, a plugin, or a scripted handoff. Six of them run on the *design* skill pack, `user-research`, `design-critique`, `design-system-management`, `design-handoff`, `accessibility-review`, and `ux-writing`, each a focused playbook rather than an open prompt.

00

Behavior analysis & backtest

The `user-research` skill ingests analytics events, session recordings, and support tickets and applies its synthesis framework, affinity mapping, impact/effort, jobs-to-be-done, to produce a behavior report with funnel drops, friction patterns, and anomalies. It then backtests them against the last round of design changes to show what actually moved.

TOOL `user-research` skill OUTPUT Weekly behavior report + design delta review
REPLACES Data analyst + UX researcher

Skill user-research behavior mode · weekly cadence

01 Pull raw signals, analytics events, session recordings, support tickets
02 Synthesize, affinity map → impact/effort → jobs-to-be-done
03 Surface patterns, funnel drops, friction motifs, anomalies
04 Backtest, diff against last design changes, measure actual lift
05 Write behavior report, ranked findings + design delta review

Runs every Monday. Report feeds step 01, every new spec starts from what the data actually showed.

01

Spec writing & feature brainstorm

A one-line brief from the CEO becomes a full spec, problem statement, scope, edge cases, acceptance criteria, risk list, inside a Claude Code skill that knows the product surface, pulls signals from the `user-research` output above, and respects the existing design system. I edit the shape of the argument, not the prose.

TOOL Claude Code skill + `user-research` context
OUTPUT PRD-grade spec with edge cases & criteria REPLACES Product manager

Skill prd-development 8-phase workflow

01 Executive summary
02 Problem statement, evidence-backed customer pain
03 Target users & personas
04 Strategic context, OKR, market, why now
05 Solution overview, high-level shape, flows
06 Success metrics, primary / secondary / guardrail
07 User stories & requirements
08 Out of scope & dependencies

Orchestrates 8+ component & interactive sub-skills across 2–4 days.

02

User journey as HTML

The spec feeds into the `user-research` skill's journey-mapping mode, which produces an interactive HTML user journey, stages, actions, emotions, pain points, and opportunity moments. It's shareable as a link, survives review edits, and becomes the single source of truth every downstream step references.

TOOL `user-research` skill → HTML OUTPUT Interactive journey map (standalone link)
REPLACES Service designer / UX researcher

Skill journey-map 6-step builder

01 Input source, conversation / file / Figma
02 Gather journey data, stages, actions, responses, tags
03 Confirmation loop, tree summary, iterate
04 Generate JSON, `docs/journeys/{slug}.json`
05 Generate HTML, styled, data-driven render
06 Completion → hand off to `/dogfooding`

Outputs a single source of truth every downstream step references.

03

Prototype from user journey + Storybook

I feed the journey HTML plus the team's Storybook into a skill that respects `design-system-management`, correct tokens, variants, sizes, and states, not new invention. It outputs a working HTML/CSS/JS prototype with routing, states, interactions, and fake data.

TOOL Claude Code + `design-system-management` + Storybook
OUTPUT Runnable HTML/CSS/JS prototype REPLACES Prototyper / front-end designer

Reference storybook what's inside

01 Foundations, design tokens (color, typography, spacing, radius, motion)
02 Components, every UI primitive with every variant × state (default, hover, active, disabled, loading, error)
03 Patterns, composed blocks (toolbars, empty states, onboarding, side panels)
04 Interaction stories, play functions capture hover / focus / loading / error paths
05 Docs, auto-generated prop tables + usage guidelines per component
06 A11y addon, contrast, keyboard nav, aria roles checked on every story

Claude reads Storybook as the ground truth for what can be reused. If a component already exists, it must be used, no reinvention in the prototype.

04

Iterate in Claude Code

I walk the prototype through the `design-critique` framework, first impression, usability, visual hierarchy, consistency, accessibility, and fix issues directly in Claude Code. I rework flows, tighten states, and fill empty / error / loading branches. Interaction design happens here now: in running code, not in Figma frames. The prototype becomes the design artifact of record. In parallel, I assess whether existing components cover the need, if they don't, I spin up the `design-system` agent to co-author new ones.

TOOL Claude Code + `design-critique` OUTPUT Reviewable prototype v1
REPLACES Interaction designer

Agent design-system 5-option menu agent

01 Prior art lookup, reuse check before building
02 Component analysis, 6-phase spec (breakdown → status → interaction)
03 Prototype builder, interactive HTML output
04 Figma component doc, spec page authoring
05 Handoff eval, readiness check before frontend

Reads `DESIGN-COMPONENTS.md` + `DESIGN-TOKENS.md` on demand; pulls Figma context via MCP. One skill per run, no phases skipped.

05

Sync to Figma → Figma Make → back to prototype

I pull the prototype into Figma and run `design-system-management` checks, correct token references, component variants, and states against the library. Figma Make then explores display states and alternative layouts at speed. Or I'll spin up a few UX direction options in Claude Design and bring those back into Claude Code or Figma to refine. Whatever wins comes back into Claude Code as a prompt to update the prototype. This is the only step where I still touch pixels directly, and it's the shortest.

TOOL Figma + Figma Make + `design-system-management`
OUTPUT Design-system-aligned prototype v2 REPLACES UI designer

Loop figma ⇄ claude-code pixel-exploration detour

01 Import prototype into Figma, via Figma MCP connector sync, layout gets regrouped + snapped to design-system tokens
02 Paste the HTML file straight into Figma Make, skips rebuild, explores states & alternates instantly
03 Generate variants, 3–5 layouts / state combos in parallel, same input prompt
04 Pick the winner, side-by-side against design-system fit, not aesthetics-first
05 Write back with prompt, Claude Code updates the prototype, keeping parity with Figma

Connector sync is the default loop. Direct HTML paste is the escape hatch when I want fast exploration instead of committed moves.

06

Handoff: Figma + prototype → engineer → Claude

Engineers receive Figma *and* a working prototype, plus a `design-handoff` spec sheet, tokens, variants and states, interaction timing, content limits, edge cases, and accessibility notes. They feed the prototype into their own Claude Code session to generate production code, treating my prototype as executable spec, not as a mockup to reinterpret. Handoff ambiguity drops to near zero because there's nothing to translate.

TOOL `design-handoff` + prototype → engineer's Claude OUTPUT Production-ready feature branch
REPLACES Design engineer / design ops

Skill wren-ux-writing copy review · en / zh-TW / ja

01 Load voice profile, `voice-profile-en.md` + `patterns.md` for tone + standard phrasings
02 Apply patterns, buttons, errors, empty states, confirmations, validation

03 Check glossary, `i18n/glossary.md` enforces consistent term mapping across locales

04 Localize, `i18n/zh-tw.md` / `i18n/ja.md` rewrites, not word-for-word translations

05 Review pass, `review.md` checks voice, clarity, consistency before handoff

Runs on the prototype copy and the eng branch. Output: polished, glossary-aligned strings in EN / zh-TW / ja, ready to ship without a round of copy edits in code review.

07

Marketing video via Remotion

Claude Code wraps the same prototype with Remotion to produce a launch video, scripted scene timing, on-screen callouts, voiceover stubs. Marketing gets a ready-to-edit MP4 without me opening After Effects. The design artifact and the marketing artifact share the same source.

TOOL Claude Code + Remotion OUTPUT Launch video (MP4) for marketing

REPLACES Motion designer / video editor

Framework remotion React → MP4

01 Compose in React/TS, scene = component, time = props

02 Parametrize, `inputProps` drive copy, data, colors

03 Preview in Studio, hot-reload timeline with real data

04 Render to MP4, CLI, server, or Remotion Lambda

05 Embed via Player, same composition inside apps (optional)

Source = the prototype. Same React components power the design artifact and the marketing MP4, no After Effects.

08

Brand: mascot, banners, teaser video

Gemini, Weave, and Lovart (free tiers only) cover the brand layer, company mascot illustrations, campaign banners, short teaser videos.

TOOL Gemini + Weave + Lovart (free) OUTPUT Mascot, banner set, teaser video

REPLACES Illustrator / brand designer

09

Dogfooding → summary → task assignment

The same journey map that drove the prototype generates a `user-research`-style dogfooding checklist (usability-test structure: warm-up → context → deep dive → reaction → wrap-up). An `accessibility-review` pass runs alongside it for WCAG 2.1 AA, contrast, keyboard, focus, touch targets. A Claude Code skill then consolidates findings, ranks by severity, proposes fixes, and drafts task assignments by owner. By Monday morning, a sprint-ready backlog I didn't hand-write.

TOOL `user-research` + `accessibility-review` + Claude Code

OUTPUT Dogfooding summary + ranked tasks REPLACES UX researcher + program manager

Skill dogfooding 6-step form generator

01 Source selection, existing journey / file / fresh

02 Analysis & conversion, regroup into 4–5 testable sections

03 Confirmation loop, structural summary, iterate

04 Output destination, Notion page / local Markdown

05 Generate the form, sections + bugs / ratings / exit criteria

06 Language, EN default, zh-TW per journey

20-minute budget per tester. Pulls from `docs/journeys/{slug}.json`, the same source that drove the prototype.

10

PM ops: Notion sync & workload math

Claude updates the Notion product roadmap against the backlog, recalculates each person's workload, and surfaces overbooked weeks before they blow up. The weekly roadmap review becomes a 10-minute review of Claude's draft, not a 2-hour spreadsheet session.

TOOL Claude + Notion MCP OUTPUT Updated roadmap + workload flags
REPLACES Producer / PM ops

Skill roadmap-planning 5-phase workflow

01 Gather inputs, business goals, customer problems, constraints

02 Define initiatives, epics with hypotheses & metrics

03 Prioritize, rank by impact / effort / strategic fit

04 Sequence, releases or quarters with dependencies

05 Communicate, Now / Next / Later narrative for stakeholders

Outcome-driven, not a feature list. Syncs to Notion so the roadmap, backlog, and workload math live in one surface.

11

External product documentation

The journey map + the `ux-writing` skill + the Claude Chrome plugin produce the customer-facing docs, help-center pages, API reference stubs, release-note drafts, plus in-product microcopy (CTAs, empty states, error messages, confirmations). The same source that drove the prototype now drives the words users read when the feature ships, with a consistent voice and tone.

TOOL Journey + `ux-writing` + Claude Chrome plugin OUTPUT External product documentation
REPLACES Technical writer / content designer

Skill user-facing-docs PRD → 4-section Markdown

01 Ask for PRD source, paste or Notion URL

02 Confirm tone, friendly / technical / formal

03 Parse PRD & apply rewrite rules, strip internal language

04 Generate four sections, Overview · How It Works · Using · Tips

Reads the PRD, rewrites from the user's perspective, drops engineering/PM framing. Help-center & manual format, not tooltips or release notes.

03. The outcome: when one person runs a system

These are my real numbers at Wren AI, proof that the pipeline above actually runs. They don't record the limits of my stamina. They record what's possible once the question stops being "how to squeeze out more design time" and becomes "how to direct a system that designs".

- 6–8FTE I'm a designer, and an army

  Before, this took 6–8 different minds, PM, researcher, UI/UX, tech writer, even motion designer. Now those boundaries collapse inside me. With AI as leverage, one person can run the whole design

production line.

- 3–4 Parallel universes with no queue

  No "waiting on design" dead time here. 3–4 fronts advance simultaneously: Agent Mode's logic, Embedded Threads' experience, Design System maintenance, all in active motion, not queued and idle.

- 16hrs/wk Time sliced by structure

  16 hours a week, this is the communication tax I pay as a "human." 40% of my time is locked to meetings, non-negotiable. Which means every remaining minute has to deliver several times its weight.

- 24hrs/wk 24 hours as conductor

  Meetings deducted, the 24 hours that remain each week are my real battlefield. Each project gets less than 6 hours. Sounds crazy? But when I'm no longer the craftsman drawing frames, when I'm the system owner directing Claude Code to execute the vision, a week's workload really does land inside 6 hours.

One Product, Three Minds

WREN AI · 2026

Preface

In AI-driven Business Intelligence, the hardest challenge was never "how to generate SQL", it was "how to earn trust." As senior product designer at Wren AI, I led the transformation from traditional conversational patterns into the Agent era. What I found: users sit between two extreme mental models, the freedom to explore, and the precision to control. This case study unpacks how three generations of architecture let us close the trust gap that a linear pipeline left behind.

My Role

Sole designer, user researcher, and PM. I owned the Langfuse + PostHog analysis, the end-to-end Agent UX, the mode strategy, and the post-launch hypothesis verification framework.

Company & Timeframe

Wren AI · 2026

01. Data Archaeology, Three Technical Strata in One Product

I analyzed tens of thousands of Langfuse traces across nearly a thousand projects from early 2026. Three distinct technological strata surfaced inside the product, more than version upgrades, they reveal three different design philosophies and the historical arc of AI's evolution.

1

Gen 1 · Ad-hoc Mode (2023–2024)

The first generation was a classic linear pipeline, one-shot in, one-shot out, like a vending machine: feed it a precise instruction, and it returns SQL. Even today, despite a failure rate around 19.5%, the 11% of active projects still living in this mode drive nearly 60% of total traffic. These power users would rather tolerate failing one out of every five queries than give up direct control over the SQL.

2

Gen 2 · Interactive Mode (2025.09)

The second generation layered memory on top of the linear pipeline, what we called True Thread Context. Users no longer had to restate background with every question; the historical context carried forward, making continuous analysis possible. That layer pushed the error rate down to a near-perfect 0.29%, and proved that conversation can absorb, and quietly correct, the small slips a pipeline makes along the way.

3

Gen 3 · Agent Mode (2026)

The third generation tore down the rigid pipeline entirely, replacing it with a multi-agent, skill-based system. Multiple agents collaborate dynamically around the user's current intent, pulling together the

strengths of the first two generations, the resilience of conversation and the precision of technical execution.

02. The Diagnosis & Decision, Decoding the Expert Paradox and Ad-hoc's Real Identity

I analyzed tens of thousands of Langfuse traces and ran deep research on 89 active users, and found the product trapped in a gap between its technical architecture and real user intent. The study revealed that the legacy linear architecture wasn't just throttling performance; it was misreading what users actually wanted.

Insight 01 · The Expert Paradox, the more expert, the more frustrated

The data exposed a counter-intuitive pattern: expert users ran into more friction than beginners. I named it the Expert Paradox. These active users drop raw schema field names straight into their prompts, their underscore density runs about 12× higher than trial users. Yet the moment they type something as precise as `BUFF_GAME_ACCOUNTS`, the linear pipeline collapses if it can't match the data model (MDL) exactly.

In fact, over 95.9% of Ad-hoc failures concentrate in `NO_RELEVANT_SQL` and `NO_CHART`. The legacy architecture had no room to clarify or guide once technical detail got messy, and that's exactly where experts live.

Insight 02 · Identity Decode, 89 users reveal what Ad-hoc is really for

To understand the motive behind the data, I ran a deep survey of 89 active users to decode their mode-switching behavior. The finding: forcing a choice before typing is a heavy cognitive tax, 25% of users couldn't even articulate the difference between the two modes.

The more shocking finding: 62.5% of users switched to Ad-hoc not to access a different analysis surface, but to "inspect the SQL logic" or "debug." In users' minds, Ad-hoc was no longer a standalone analysis entry, it had become a Debug Lane for verifying results. The survey also overturned the "keep old modes as a fallback" assumption: only 2% of users switched modes because of system stability.

The pivot · From "mode switching" to "control sovereignty"

The data pointed to a clear direction: users wanted control, not a mode switch. Rather than patch a leaky linear pipeline, we merged the two generations outright. The new Agent Mode inherits the conversational resilience of Interactive, while turning Ad-hoc's debug needs into a built-in skill, so experts get automated analysis and keep absolute sovereignty over the underlying data logic.

03. The Solution, Fusing Two Generations of Experience

Rather than simply killing old features, we performed a deep technical and design integration of two generations of hard-won experience.

Drawing from Interactive Mode: Conversation builds patience

From Interactive Mode, we learned that conversation gives users the patience to tune results. Agent Mode uses a series of Checkpoints (pre-selected answer panels) to turn data exploration from a one-shot attempt into a rhythmic, output-guaranteed loop. When users submit precise but ambiguous fields, Agent no longer throws `NO_RELEVANT_SQL`; it asks to clarify, converting potential crashes into a successful guided flow.

Transforming Ad-hoc Mode: Debug experience as a built-in skill

The transparency users previously had to switch modes for, we turned into a core capability of Agent through a Skill-based architecture. Addressing that 62.5% demand, we built the debug experience directly into how Agent operates. Users can now customize skill execution parameters on demand, and every SQL generation and reasoning path is fully transparent and queryable in natural language.

[interactive demo omitted in PDF]

Guided Discovery

Guided Discovery: Defining the Shape of the Question

Users often arrive with a clear goal but struggle to phrase the precise question. When a business user enters with a vague intent like "explore this data", Agent's job is no longer to guess, it's to guide.

I designed a progressive confirmation flow: at every fork that would change the outcome, the system surfaces a single-select panel (analysis type, time dimension, segmentation). Each panel is a Checkpoint that moves the analysis forward, not a dead-end that blocks exploration, and a `Skip` option preserves the freedom experts need alongside the guidance newcomers want.

[interactive demo omitted in PDF]

Enhance Thinking Steps

The Strict Auditor inherits Ad-hoc's control, as a transparent trail, not a mode.

In this redesign, we baked transparency into the product's DNA.

Streaming Logic: Through Thinking Steps, the agent's actions stream out as primitives. Beyond the visual spinner → checkmark, the SQL and tool calls beneath are fully exposed, giving you a god-view of the AI's reasoning.

Skill, authorable precision: We dissolved Ad-hoc Mode entirely and replaced it with the Skill system. You author your own context retrieval and schema mapping logic, turning a heavy mode into a lightweight, reusable skill set, making conversation itself the development surface.

Official Skills

Defining Official Skills, from feature buttons to atomic capabilities.

In the previous linear architecture, core capabilities like report generation, data evaluation, and document parsing were scattered across the UI as buttons or dedicated modes. That bloated engineering cost and broke user flow. We redefined Official Skills, packaging this complex back-end logic into atomic capability modules.

The deepest design implication of this shift is this: Skill lets users traverse the entire system through conversation alone, interacting with every capability systematically and safely. When every capability collapses to the same trigger grammar, natural language becomes the product's only entry point. On that foundation, three core advantages come into view.

1 · Unified routing, turning UI clicks into natural-language commands.

Tasks that used to require clicking around the interface now route through Skills.

- `generate-report` no longer needs a "generate report" icon, the agent triggers it from context or the user invokes it via command.

- `analyze-data` and `sql-queries` split "answer the question" from "write the SQL," letting the agent know exactly when to explore versus when to fetch a precise number.

2 · Bridging document handling, doc & pdf as capabilities.

Heterogeneous data (spreadsheets, PDF, Word) has always been a BI pain point. The `pdf`, `doc`, and `spreadsheet` Skills decouple parsing logic from the main app. When users drop a `.zip` or mixed-format files, the agent stops reading chaotically and dispatches to the right Skill instead.

3 · Turning patterns into reusable knowledge assets.

Adding domain rules (Evals) or data patterns to the knowledge base used to mean tedious manual setup. Now that logic packages inside a Skill. Beyond easier maintenance, this gives user-authored Skills a benchmark: Official Skills act as a reference implementation that teaches users how to embed their own Eval logic or workflow into the agent's DNA using the same pattern.

Official Skills coverage, defining the agent's core boundary.

We curated six Official Skills as the agent's base capability layer:

Skill ID	Purpose & application
`analyze-data`	The analysis core. Triggered when users want to explore trends, fetch specific numbers, or visualize results.
`sql-queries`	Precise data retrieval. Focuses on writing and running SQL for direct answers.
`generate-report`	Narrative output. Assembles scattered insights into a complete report with charts and summaries.
`pdf` / `doc`	Unstructured extraction. Pulls text and tables from PDFs and Word docs into the analysis pipeline.
`spreadsheet`	External data conversion. Imports and transforms CSV/Excel for cross-source analysis.

Why it matters.

Turning features into Skills is, above all, a cognitive-load reduction. Users no longer have to memorize "where is this button?", they just focus on "what am I trying to do?" For the product team, it makes capability expansion radically lightweight: shipping a new AI evaluation model or data-cleaning routine means publishing a new Skill, not refactoring complex UI flows. This "decouple features, aggregate capabilities" architecture is exactly the technical asset that lets Wren AI iterate fast in the agent era while preserving audit-grade transparency.

[interactive demo omitted in PDF]

04. Closing Reflection, Beyond the Artifact

What we ultimately shipped was the soul of the Strict Auditor carried inside the body of a lightweight conversation.

SQL inspection and logic debugging are no longer foreign objects living outside the experience, they become reasoning receipts available on demand. Removing Ad-hoc was an experiment in trust migration: when we successfully blocked errors at the intent stage and surfaced a smooth, transparent trail at the execution stage, users moved from being tedious inspectors to composed governors. The interface got simpler, yet users' sense of control over the AI reached a depth we hadn't been able to deliver before.

While designing this product, I found myself wondering: if a designer no longer draws, is she still a designer? How do we measure the "goodness" of a creator when the traditional craft is automated? When the physical or digital "object" loses its primacy, what remains of the designer?

These are the questions that haunt me as I build inside the realm of AI. In an era where the interface has been reduced to a simple chat bubble and a text box, what is left for us to do?

The Death of Process, the Birth of Intent

While in Bali, I encountered the work of Anthropic designer Jenny Wen, who famously declared that the "design process is dead." That sentence sent shockwaves through the design community. Yet, caught between the extremes of quiet meditation and the relentless monitoring of user data, I arrived at a realization: we become designers because there is something we burn to create, a world we desperately want to see exist.

The tools have multiplied and the noise has grown deafening, but the core philosophy remains unshaken. We must constantly ask ourselves:

- What is the message we are trying to convey?
- What is the fundamental value we offer the user?
- What do we want the user to carry away after the interaction is over?

Beyond the Artifact

The "design object", the interface, the icon, the layout, is merely a vehicle. It is a waypoint in a larger journey. The essence of design has never been about the pixels; it has always been about influence.

A mentor once told me: "A designer must never give up on dialogue. The moment a designer stops communicating, design dies."

As AI designers, our role has evolved. We are no longer just building tools; we are crafting the medium for dialogue itself. We design AI to converse with humanity, to inspire, to assist, to empower. Our ultimate goal is to encourage people to use AI to create things that influence others, sparking a virtuous cycle of creation and conversation.

Embedded Threads

WREN AI · 2026

Project Overview

While building Wren AI as a conversational BI product, what customers kept asking for wasn't another tab to open, it was a way to talk to their data right where they already were: a Notion sidebar, a phone modal, a corner of a Tableau dashboard. **Embedded Threads** is the answer the team and I designed for those native contexts; from a 1024px desktop split to a 393px mobile screen, I made sure the product still carried Wren AI's core value even at the tightest possible spaces.

My Role

As the sole designer, I owned the planning of this embedded experience end to end, covering configuration logic, device adaptability, error guidance, and the operational details once it's integrated into another product, making sure it performs well in every context.

Company & Timeframe

Wren AI · Dec 2025 – Jan 2026 · 3 weeks

01. Product Challenge & Market Strategy

In BI, there's a real threshold: a tool you have to jump to a separate tab to use is destined to be forgotten.

We found that most of Wren AI's core users are in a "multitasking or on-the-move" state: reading Notion on a commute, swiping through a dashboard on their phone during a meeting. For them, **an embedded experience isn't a bonus**, it's what decides whether the product gets into the decision-making moment, or doesn't survive there at all.

To reach users fast, we avoided reinventing the wheel and instead studied industry benchmarks like Metabase and Tableau closely, sorting out which features should follow industry standards to lower the learning curve, and which deserved aggressive simplification.

In the end, we committed to a **plug-and-play install model**, a clean snippet customers drop in for fast deployment, deliberately walking away from a heavyweight SDK framework. Because we knew what users actually want is to weave data conversations seamlessly into their workflow, not to maintain another integration project.

02. Naming the Architecture: Two Modes, Zero Decision Ambiguity

To keep product logic unambiguous, the team and I split the embed architecture into two distinct behavior modes: **Public Embed** and **Identity Verification**.

Once the architecture has a clean name, the harder questions further down, permissions, refresh policy, empty states, start answering themselves. The underlying philosophy: "shared information in a Notion doc" and "personalized data inside an internal system" are fundamentally different product scenarios. Try to cover both with one mode and you make both worse.

- **Public Embed**, built for Notion sidebars, Tableau panels, and public-facing info pages. A "shared conversation" model where every visitor reads and contributes in the same context, with a one-minute

deploy.

- **Identity Verification Embed**, built for internal systems and apps with real account systems. Integrates with the host product's auth so identity flows through and each person sees only their own conversation history, full capability, full privacy boundary.

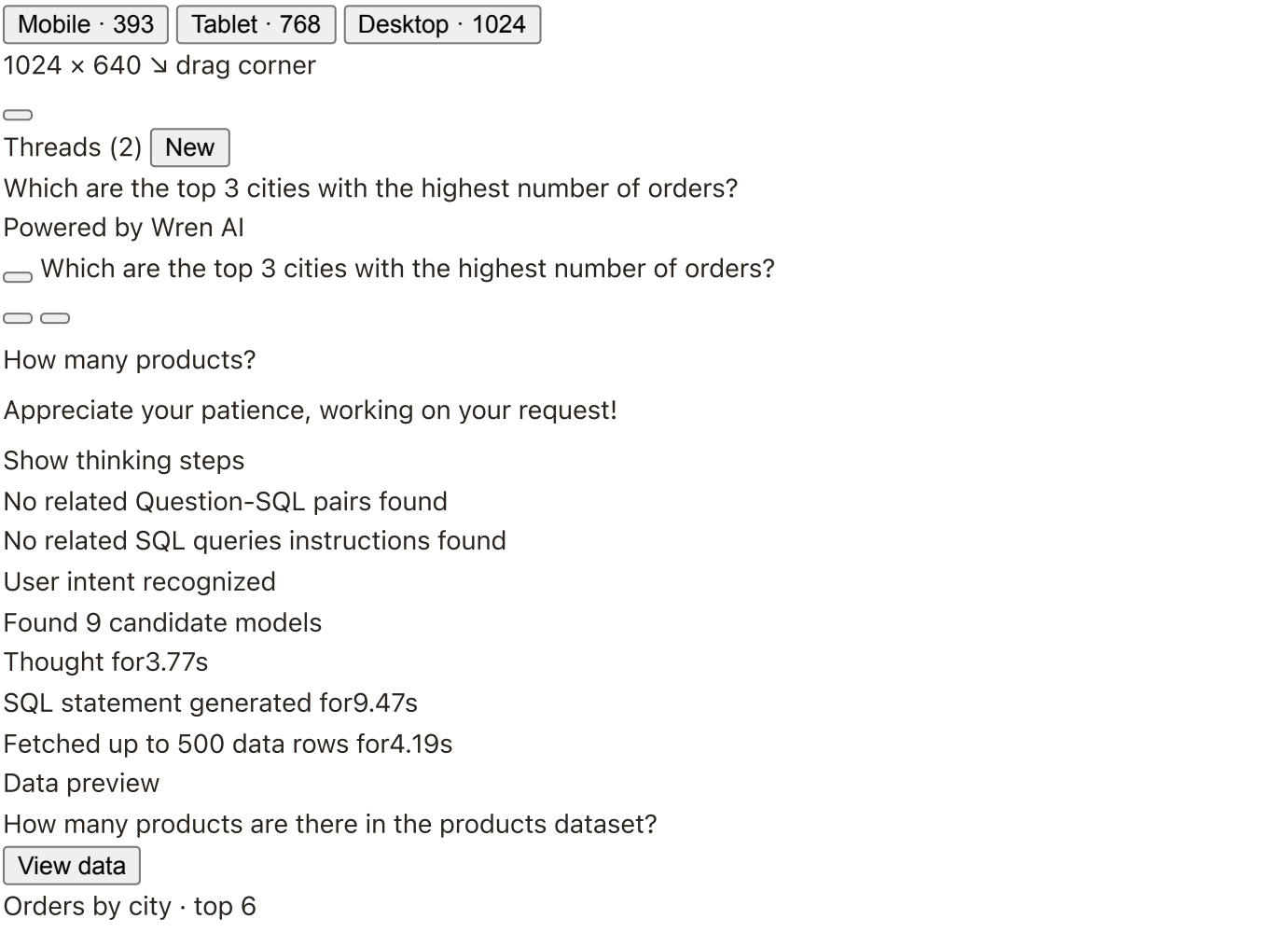
03. The Design Challenge: Constraint-Driven, Inside-Out

The hardest part of this project, keeping the experience consistent across radically different host products. The challenge we took on: treat Embedded Threads as its own surface, not a smaller version of the main app.

The team and I designed inside-out, anchoring at the tightest constraint, 393px phone width, instead of starting from desktop and shrinking. That meant the interface had to feel native whether it was tucked into a narrow Notion sidebar or sliding up from a phone bottom-sheet. If a design works at the tightest constraint, it works at every wider one. An embedded product that only survives on a roomy desktop doesn't belong in the way modern teams actually work.

RWD playground, drag the corner and watch it collapse

Drag the corner or switch presets and you see three real surfaces: at 1024px the sidebar and chat sit side by side; at 768px the layout tightens without restructuring; at 393px the embed slips on iPhone chrome — notch, home bar, sidebar collapsing into a hamburger drawer, and the chat flow, question, thinking steps, data preview, chart, answer, restacks at phone rhythm. The input stays pinned to the bottom so a follow-up is always one tap away.



Ask to explore your data

04. Design Is Navigation Anchored to the Human

This Wren AI build felt more like an experiment about boundaries. We let go of our attachment to data models and moved the design's anchor into a subtler territory, the human scenario. The moment we started asking "where is the user, what screen are they looking at, how much attention can they spare or spend," what had been rigid UI decisions began to settle into a kind of self-organizing order.

It made us reconsider what data really is. Maybe it shouldn't be a destination passively waiting to be reached, but a flowing state that ripples with context and can be summoned at any moment. Through this work, we found that when design aligns precisely with the slice of life the user is in, data stops being a burden and becomes a reliable companion ready to act alongside them. The best design doesn't create a new world; it slips quietly into the user's own.

Closing the Loop

WREN AI · 2026

Project Overview

To give enterprises greater control over their AI applications, my team and I designed **Thread Tracing**, a governance module that lets AI administrators observe organization-wide interaction data from a holistic perspective. Grounded in Google's PAIR framework, the system transforms scattered user feedback into structured, visual intelligence. Administrators can proactively identify model deviations and convert feedback into actionable instructions, closing an autonomous loop of **monitor, feedback, and optimize**.

My Role

PM, sole designer, and user researcher. I owned the feedback taxonomy design, the dashboard information architecture, the feedback-to-instruction pipeline, and the PAIR framework application.

Company & Timeframe

Wren AI · 2026

01. Our Mission: Building an Evolutionary "Harness" for Enterprise AI

As Wren AI moved into enterprise environments like TSMC, we found that what administrators feared most wasn't AI "not knowing the answer"—it was AI "giving the wrong answer with no one knowing why."

Today's AI runs like a black box, facing two governance challenges:

1. **Broken Perception**: A user hitting "thumbs down" is like a patient crying out in pain, but the administrator can't pinpoint which nerve is the problem.
2. **Open-Loop Governance**: The system lacks any automated validation mechanism, leaving administrators stuck in a "fix A, break B" blind-man's-buff loop.

Harness — The Evolutionary Scaffold Around the Model

A confidently-deployable, deterministic enterprise AI system must be wrapped in a defensive and amplifying layer called the Harness. It gives the model capabilities it doesn't natively have: a **toolbox (Tools)** for reaching the outside world, **memory & state (Memory & State)** that persists across tasks, a **safe sandbox (Sandbox)** for running dynamic code, and a self-correcting **feedback & evaluation loop (Feedback Loops)**.

As Senior Product Designer, my team and I were tasked with designing the last critical piece of this Harness: an interactive feedback system (Feedback Loop) that moves AI from "repeated execution" toward "closed-loop evolution."

02. Problem Definition: Three Systemic Symptoms Inside a Closed Ecosystem

Through continuous observation of user behavior and query patterns, we distilled the enterprise's complex pain points into three recurring "failure modes" — the defensive lines we had to break through by design:

1. **Noisy Signal: Interface Failure and the QA Bottleneck**
   The existing Thumbs Up/Down was buried at the edge of the interface, leaving user engagement extremely low. Even when users did leave negative feedback, it was usually a vague short phrase like

"this answer is useless" — too ambiguous for AI to extract any actionable optimization signal. For the IT team, this directly turned into a "Manual QA Bottleneck." To make sure each update hadn't regressed, engineers had to manually replay large question sets, slow, expensive, error-prone, and chaining model iteration speed to human labor.

2. **The Black Box's Regret (Transient Failures): Observation Gap and Outcome Black Hole**
   While the team internally observed traces via Langfuse, this capability was never "productized" for enterprise administrators. Companies couldn't analyze or optimize errors on their own and relied heavily on post-sales support. More fatally, there was a "historical-data gap," administrators couldn't track outcomes, couldn't tell which optimization actually worked vs. which one just got lucky, leaving every round of investment a blind bet.
3. **Misaligned Correction: Baseline-less Fixes and Blind Releases**
   Without a standardized Golden Dataset for blind testing, administrators adjusted Prompt or Schema based on intuition and repeatedly fell into a whack-a-mole patching cycle: fix scenario A, accidentally break scenario B. Without quantitative regression testing, the team had no scientific "release criterion," no one could confidently decide whether a new version was actually better than the last, or whether it was safe to ship.

03. Solution, The Feedback Loop We Designed

Why Feedback Loops Matter

Referencing Google's PAIR framework, feedback is more than opinion collection, it's the core engine driving system evolution:

1. **Model Optimization (Ground Truth)**, Transforming user feedback into high-quality labeled data that precisely calibrates model errors.
2. **Personalized Perception (Personalization)**, Continuously learning individual preferences to craft tailored interaction experiences.
3. **Trust & Agency**, Empowering users to intervene in decisions, shifting from passive acceptance to active guidance and building deep trust.

Balancing Dual-Track Collection

In our design, we adopted a parallel implicit-and-explicit strategy to ensure both breadth and depth of data:

1. **Implicit Feedback, Data Breadth:** Automatically monitoring clicks, dwell time, and ignore behavior as low-friction signals. Yields massive behavioral samples, but environmental noise can mislead.
2. **Explicit Feedback, Intent Depth:** Capturing clear intent through ratings, thumbs up/down, and structured error reports. Higher interaction friction, but delivers the purest optimization guidance.

Based on the above, my team and I designed Thread Tracing as a four-stage closed loop, each stage directly targeting a gap we identified.

01
Structured Intake
Users flag an AI error, pick an error type, and the reasoning trace is auto-captured.
trace + error type
02
Error → Test Case
Admin diagnoses the failure and normalizes it into a repeatable test case.

normalized case
03
Guided Correction
Compare wrong vs. right paths and target the fix at MDL or instructions.
updated MDL
04
Regression Verification
Re-run the full test bank to confirm the fix passes without breaking existing answers.
↻ verified behavior feeds the next cycle
01

Structured Intake, Turning Noise into Signal

Feedback should blend naturally into the workflow. Previously the thumbs-up / thumbs-down icons sat almost invisibly in the action row, so few users ever pressed them. We made the ask explicit, extended the Yes / No click into finer-grained tags (SQL generation failure, retrieval error, instruction non-compliance, summary error, SQL-Pairs deviation), and auto-captured the AI's reasoning trace at the moment of failure, preserving the "crime scene" that had previously vanished.

Decision 01 · Make the ask explicit
Pull the question into the line of sight, not the action row.

The old design buried feedback in the generic action row, a tiny thumbs-up / thumbs-down wedged between copy, share, and regenerate. Almost nobody pressed them. We pulled the question into the answer itself with an inline `Was this result helpful?` prompt, pinned directly below every response. The Yes / No click stays low-commitment on purpose; the structured intake only unfolds when a user actually has something to report. Lifting the ask from chrome to content was the single biggest behavioral lever in this chapter, response rate climbed +45% once the prompt was impossible to miss.

Before · icons only
After · explicit prompt
Decision 02 · Tag taxonomy, not free text
Route noise into five named failure modes the AI team can already act on.

An early open-text version of this form taught us that unstructured feedback fragments into paraphrases of the same problem, *"the SQL is wrong"*, *"it used the wrong table"*, *"the query broke"* all meant one thing but read as three. We mapped the five real AI failure modes, `SQL generation failed`, `Failed to retrieve data`, `Failed to follow instructions`, `Incorrect AI summary`, `SQL–Pairs deviation`, onto a fixed chip set. Users tap; the system aggregates. Signal survives the translation, and the admin dashboard can finally count apples against apples.

Decision 03 · Depth on demand, not by default
Keep the form lean; let one tag earn the right to unfold.

A long form hurts completion rate; a short form loses specificity exactly where it matters most. The compromise: the modal stays lean by default, but when a user picks `Failed to follow instructions`, the one tag where *"which rule"* changes everything, a second dropdown reveals every project-level instruction currently configured. `Other` exists as a last resort, deliberately positioned as the least actionable option. The admin either gets a named rule to fix, or a clear signal that the instruction taxonomy is missing coverage.

Decision 04 · One shape, two contexts

Swap the vocabulary, preserve the muscle memory.

Answers and charts fail for different reasons, so they can't share one tag list, but they have to share one mental model, or users stop trusting the form after the second surface. Every feedback modal keeps the same three-part shape: **rate → tag → optional description**. Only the tag vocabulary swaps. A user who gave feedback on a broken SQL answer already knows what to do the first time a chart renders with the wrong axis, even though the specific options are different.

02

Monitor Quality, From Count to Cause

The feedback modal was the producer; this dashboard is the consumer. Project owners open `Settings → Project → Thread tracing` and land on a single screen that has to answer two questions fast, is the AI getting better or worse this week, and what's the dominant failure mode right now. Everything else is a filter.

Decision 01 · Trend first, taxonomy second
Lead with the line; let the bars explain.

Admins don't want to open on a list of failures, they want to know if the needle is moving. The hero of the dashboard is a success-metrics chart (positive vs negative feedback, by day), with the failure-breakdown bar chart sitting directly underneath. The order matters: you glance at the trend line to decide whether last week's fix actually helped, then drop to the bars to see which category is spiking. Putting the breakdown first would have inverted the question into "what's failing?" before the admin had even decided whether anything needed fixing.

Decision 02 · Filter by who reported, not only what happened
Put identity on the filter shelf next to time and type.

Early prototypes filtered only by date range and trace type (`Answer` vs `Chart`). That wasn't enough once we talked to cross-team admins, marketing's failure profile looks nothing like finance's. We promoted **Reported-By** (specific members and user groups) into a first-class control rather than hiding it behind an advanced options menu, so the same bar chart can answer *"which team's queries is the AI struggling with?"* in two clicks. On the table below, a thumbs-up / thumbs-down toggle lets an admin zoom straight to explicit negative signal when they're hunting for the next correction.

Decision 03 · From count to cause in one click
Collapse the table; let every row promise a deeper view.

The thread table is the bridge between *"a number went up"* and *"here's the actual broken query."* Each row collapses to Thread ID, timestamp, trace type, and sentiment. Expand it and the AI's suggested failure reasons appear inline, so admins can decide whether a full dive is worth it before opening the trace drawer. `Action → View` is where the reasoning trace, metadata, and SQL live, deep information, but gated behind the admin's intent rather than served up front.

Decision 04 · The drawer as a crime-scene brief
Everything you need to judge the failure, without leaving the table.

The drawer opens at the moment of decision, you've scanned the expanded row, the suggested failure reason looks worth investigating, and you click `View` . What you need now isn't a new page: it's a focused brief. The drawer surfaces creator identity and the reporter who flagged it (separating who ran the thread from who noticed the error), the AI's run status (distinguishing what the machine recorded from what the user perceived as broken), the issue tags the user chose, and the thread name that gives the failure

natural-language context before the SQL appears. Every field earns its place, remove the reporter and you lose the cross-team signal; remove the thread name and the SQL reads without context. The drawer is designed to be read in thirty seconds and decided on.

03

Error → Eval Suite, Closing the Loop on Failure

The feedback modal captures the signal; the trace drawer names the failure. This chapter is where that named failure becomes a repeatable test. When an admin converts a diagnosed thread into an Eval question, they're not writing a test case from scratch, they're turning a real user complaint into a permanent benchmark that runs against every future correction.

Decision 01 · Ground Truth SQL as the anchor
Pin "correct" before you measure "wrong."

A test case without a ground truth is just a question. The Eval suite requires admins to pair every natural-language question with a verified SQL, either hand-authored or AI-generated and approved. This pair is the unit of measurement. Once locked, it becomes the objective standard the AI is scored against, not a human's memory of "it used to work," but a reproducible, version-controlled definition of correct. The `Preview Data` button closes the loop at definition time: you know the SQL is valid before it ever becomes a benchmark.

Decision 02 · Two-step creation separates SQL accuracy from narrative accuracy
Correct SQL and correct output are two different questions.

Step 1 establishes the SQL anchor. Step 2 establishes what the answer should actually say, key findings, data sources, conclusions. Separating these steps forces admins to think about correctness at two levels: *"Did the AI write the right query?"* and *"Did the AI surface the right insight from it?"* A query can return the right data and still produce a misleading summary. The two-step model makes that distinction explicit, so regressions in either layer are caught independently.

Decision 03 · The benchmark result is a diff, not a verdict
"Failed" is a starting point, not a finding.

The assessment view shows Pass / Fail alongside a score reason (*"Column count mismatch: generated is missing 1 column"*), a side-by-side SQL diff, a data preview comparison, and the AI's reasoning timeline. Each element answers a different diagnostic question: the score reason says *where* the divergence happened, the SQL diff shows *which clause* caused it, the data preview shows whether it mattered in practice, and the reasoning timeline reveals whether the AI misread the question or misbuilt the query. Admins leave the view knowing not just that a correction failed, but where to aim the next one.

04

Guided Correction, The AI Knows What to Change, Not Just What Failed

The Eval suite names what failed. The AI Advisor takes it from there, running a diagnostic pass on the benchmark failures, staging specific knowledge-base changes, and verifying the result before anything ships. Correction becomes targeted instead of instinctive.

Decision 01 · Select failures, not symptoms
Diagnosis starts with the benchmark, not a description.

Admins don't describe the problem to the advisor, they select the failing questions directly from the benchmark result. The advisor takes those as input and runs its own diagnostic pass across the knowledge

base. The selection step matters: it scopes the diagnosis to confirmed failures, not hypotheticals, which keeps staged suggestions tightly coupled to observed regressions rather than guesswork.

Decision 02 · Staged changes, not one-click fixes
Every suggestion is reviewable before it becomes a commit.

The advisor surfaces schema adjustments and instruction edits as staged proposals, one change at a time, with edit and deselect controls on each. Admins can review the logic, modify the wording, or remove an item entirely before anything is applied. The separation between diagnosis and apply is intentional: it keeps human judgment in the loop on changes that touch the AI's core knowledge model, where a poorly scoped instruction can create new regressions while fixing old ones.

Decision 03 · Verify before apply
The benchmark decides whether the fix is ready.

Before staged changes are applied, the advisor runs a verification pass against the same benchmark questions. A Before / After column shows the per-question outcome, which failures resolved, which remained, and whether anything that previously passed has regressed. Admins commit only when the verification run agrees the changes are safe. This gate means every staged fix has been tested, not just reviewed.

05

Regression Verification, Pass on Every Round, Not Just This One

The advisor's verification pass only re-tests the questions it set out to fix. But a schema edit written to patch one failure can quietly break a question that was already green. Before anything ships to the AI system, the full benchmark runs one more time, every question, not just the failing ones, and only a clean sweep unlocks the final apply.

Decision 01 · Full apply, guarded by a full re-run
The benchmark is the final approver, not the advisor.

Apply to AI system is intentionally the end of a two-gate process. The first gate is the advisor's Before/After on the failing subset; the second gate is a full-bank re-run that surfaces any new regressions introduced by the staged edits. Only when both rows read clean does the button activate. What ships isn't a suggestion an admin approved, it's a correction that the benchmark itself has signed off on.

06

The Loop Closes, The Record Becomes the Next Starting Line

With apply complete, the benchmark run becomes the permanent record for the cycle. Accuracy delta, an *Improvement: Completed* stamp, every schema field the advisor touched, every instruction it added, all stitched to one timestamp. The loop doesn't just close; it leaves a signed receipt the next cycle reads from.

Decision 01 · The completed run is also the audit trail
One timestamp, one signed summary of what actually shipped.

The benchmark result page carries three things at the top, the accuracy score, the *Improvement: Completed* label, and an expandable Advisor result that inventories every schema adjustment, every Specific or Global instruction, and a per-question verdict. An admin returning three weeks later can reconstruct why the AI behaves the way it does without asking the original author. For the next correction

cycle, this record is also the baseline: the next full re-run compares against this version of truth, not against memory.

04. Design Pivot, From One-Shot to Iterative

The first AI Advisor I shipped assumed correction could be a single transaction. On a failed benchmark question, the admin opened a right-side drawer, read the score reason, and saw two things directly: a proposed schema adjustment with an `Apply adjustment` button, and a recommended instruction with an `Add to instruction` button. One click per row committed the change. That was the entire user flow, no verification pass, no full re-run, no trace of whether the fix actually held across the rest of the bank. The advisor spoke, the admin clicked, the drawer closed.

BEFORE · One-shot AI advisory drawer, Apply and Add, then the flow ended.

Testing with real eval sets proved that mental model wrong. A schema note that fixed one failure often introduced a new one elsewhere, and the admin had no way to see it until a user complained again. Correction isn't a transaction, it's a cycle. That realization reshaped every screen that came after: we stopped designing a linear *diagnose → fix → done* flow, and started designing for repeated rounds of test, adjust, and re-verify, with the benchmark sitting between each cycle as the arbiter of whether a fix actually shipped.

The second version replaced the single click with a loop. Every staged schema modification and every new instruction now passes through a Before / After verification run, the same failing questions, re-executed with the edits, shown side by side so admins can see exactly which failures resolved and whether any passing question regressed. Changes aren't overwritten; they accumulate. Each round adds to a growing stack of schema adjustments and instructions that the advisor has already confirmed safe, and `Re-run verification` lets the admin add another edit and test it against that stack before anything reaches the knowledge base. A correction is no longer final after one click, it's final after the benchmark stops surfacing regressions.

AFTER · A multi-round Before / After verification loop, every schema modification and instruction earns its place only after repeated re-runs.

This was the most critical, and most labor-intensive, part of engineering collaboration. When each correction round may trigger new side effects, the interface can no longer be a one-way conveyor belt. We had to design every step for "what if this needs to be done again": error classification needed to support re-tagging, the test bank needed incremental expansion rather than full replacement, and correction results needed immediate comparison with previous rounds rather than simply overwriting them.

This mindset shift also changed how we presented AI reasoning paths to administrators. In the initial linear model, we only needed to show "here's the error, here's the fix." But in an iterative model, administrators need to understand why the same issue recurs, which previous corrections may have introduced new problems, and which areas still haven't been covered by the test bank. Information architecture evolved from a simple diagnostic report into a multi-round correction history with contextual comparison.

The most valuable lesson: once we accepted that correction is a cycle rather than a destination, design decisions became much clearer. Every feature needed to answer one question, "Does this help the next iteration go more smoothly?", rather than "Does this solve the problem once and for all?"

05. Results & Reflections, From Passive Observation to Active Governance

Thread Tracing turned what had been scattered, headache-inducing user errors into a core data asset that drives the product's evolution.

A Systemic Shift in Collaboration Efficiency

Building the Test Set mechanism standardized the language my team uses for AI troubleshooting. It dissolved the gap between engineering and design over *what the error actually was*, shrinking a correction cycle that used to take days of back-and-forth down to minutes-long exercises of targeted diagnosis and fix.

Redefining the Designer's Role

This project reshaped how I see my role. In an AI product, a designer's job reaches beyond the surface of the interface, it becomes the work of building the governance logic. I wasn't just fixing a bug; I was building a framework that made the AI's black-box reasoning transparent and controllable.

Notification Framework

GOOGLE PIXEL · 2025

AI

Classifier

· 2025

01.

6

- , PM
- ,
- ,

02.

1,

6

2,

3,

PM

03.

1

8

2

4

3

4

4

AI

PRD

1

,

1.
2.
3.
4.
5.
6.
7.
8.

2

,

3

,

04. AI , Gemini Gem

1

2

(AI Addition)

3

4

5

→ →

05.

-
-

-
- AI

- PM
- MVP

AI

Rezio Design System Overhaul

KKDAY REZIO · 2024–25

Quick Product Intro:

The Design System Rebuild aimed to unify KKday Rezio's fragmented UI components and optimize collaboration between design and engineering. Our goal was to reduce visual inconsistencies, accelerate development, and build a scalable foundation for future product expansion.

My Role:

As part of the design team, I co-led the rebuild. We trimmed the system down by 33% (from 60 to 40 variants) while adding key pieces like a multi-date picker and form filters. Beyond components, I pushed for UX guidelines, built a Figma library, and worked closely with engineers to set up workflows for version control, merging, and testing. By integrating reusable parts early and using AI-powered prototyping, we sped up iteration, cut rework, and made the system easier to maintain.

TimeFrame:

2024–2025

01. Achievement

33% Fewer Variants

Streamlined the design system by reducing component variants by ~33% (from 60 to 40).

90% System Completion

Achieved over 90% completion of the system within one year, with ongoing improvements underway.

Aligned Workflows

Set up cross-functional collaboration workflows with engineering for consistent version control, merging, and testing.

02. Where We Started

Project Background:

This design system rebuild didn't start from scratch. One of the previous designers had attempted to establish one, but due to a lack of ongoing communication with engineers, it was never fully adopted. In addition, over time, the old system became increasingly outdated and revealed multiple issues, such as insufficient color contrast that failed accessibility checks, an excess of similar colors causing visual inconsistency, and numerous button styles lacking standardization.

Our Goal:

Rebuild a scalable system that fixes inconsistencies, makes components easier to use, and fosters stronger designer–engineer collaboration.

03. What Was Broken

When Past Designs Turn Into a Tangle, What's Next?

We retired outdated files, kept valuable insights, and rebuilt components. Like fixing a ship mid-voyage, changes had to be gradual, not overnight.

A real design system isn't just files in Figma, it's shared understanding between design and engineering. Our old system lacked alignment, slowing updates and hurting consistency. The new system focuses on fit for the team, usability, and sustainable collaboration.

04. Workflow

Through multiple iterations and real project trials, we refined a collaboration process with engineering that ensured new components were not only well-designed but also technically feasible, scalable, and easy to maintain:

Step 1, Observation & Requirements Gathering

- Continuously monitor component usage across projects and document any inconsistencies or missing patterns.
- Gather feedback from the team to identify pain points and new component needs.

For example, in the original design, selected time tags would push the input field downward, so we needed a multi-select picker that allowed users to display multiple times directly within the picker. In addition, the single-select picker had efficiency issues, it required users to click repeatedly to complete the selection, resulting in unnecessary actions.

Step 2, Discussion & Direction Alignment

- Review requirements with engineers, discussing functionality, interaction details, technical constraints, and scalability.
- Decide whether the component should be added to the Design System as a shared, reusable asset to support cross-project usage and easier maintenance.

For example, when we identified the need for a multiple time picker, we worked with the engineering team not only to communicate the strength of the demand and its scope of use, but also to ensure it could, while reusing parts of the single-select picker design, display multiple selected times directly within the picker without breaking the layout.

Step 3, Prototype & Usability Testing

- Create interactive prototypes and validate feasibility with both design and engineering teams.
- Conduct internal usability testing and iterate to converge on the best solution.

For example, when designing the multiple time picker, we built an interactive prototype and used Lovable for AI-powered prototype testing. This allowed us to quickly simulate user flows, gather instant feedback, and refine the design before moving to final implementation.

Step 4, Final Design & Development

- Finalize design specs and deliver a complete handoff package to engineering.
- Maintain close communication during development to resolve implementation gaps quickly.

For example, during the multiple time picker implementation, we used Storybook to review and validate the component. This allowed us to check functionality, interactions, and visual consistency in an isolated environment before integration, ensuring the final build matched the design specifications.

Here's the final version we arrived at through prototyping exploration:

1. When typing in the input field, the list automatically scrolls to the matching number, checks it, and pins it to the top.
2. When selecting by scrolling, the item is checked in place without being pinned to the top.
3. After adding, the input field is cleared.

05. Our Design System Structure

Visual Style

Visual Style is the foundation of a product's look and feel, defining the overall aesthetic. It includes colors, typography, spacing, icons, etc. Below is our token naming hierarchy, using color as an example:

Raw Value
#005ABE
Primitive
Cozy Blue-6
P
600
Semantic
color-background-primary
Button

Components

Components are the reusable building blocks that bring consistency to the interface. They include buttons, input fields, menus, cards, etc. Our components are organized into categories such as General (buttons, typography, etc.), Layout, Data Entry, Data Display, and Feedback.

From an initially unstructured and chaotic state, we reorganized them with clear classifications and rebuilt the library from scratch. It took a year to reach over 90% completion, and the system continues to evolve.

Patterns

Patterns are repeatable solutions to common interaction scenarios, defining not just the flow but also how the interface responds to different states. They guide users through tasks and ensure a cohesive experience across the product.

In our system, components are designed to handle six key states, Read Only, Disabled, Error, Hover/Pressing, Default (Empty), and Filled, ensuring clarity and consistency in every interaction, giving the team a shared playbook for solving recurring design challenges.

Priority	Status	Usage
1	Read Only	The field is in a read-only state, and the entered values or functionalities cannot be changed. This applies to viewing the results of filled status fields. If the field is Default and has no value, it will display as Disabled in Read Only state.
2	Disabled	The field is in a non-editable state, applicable when the field has no practical function.
3	Error	Field error state, applicable when there is an error in the field.
4	Hover / Pressing	Field hover/pressing state, applicable when the field is highlighted during interaction.
5	Default / Filled	Field default and filled states, applicable to general status fields. If the field is filled and has a value, it will display as Read Only in Read Only state. If the field is Default and has no value, it will display as Disabled in Read Only state.

Guidelines

Guidelines are the rules and best practices that guide design and development decisions. They cover areas such as component usage contexts, accessibility standards, and naming conventions. Below are the

guidelines for using different buttons side by side, ensuring visual harmony and clear hierarchy in mixed-button layouts:

Consistency
Do: Use the same type of button in the Action Bar to maintain consistency. Try to unify the text descriptions and icons for the same type of button.
Don't: Avoid using different button types in the Action Bar that create a patchwork effect, leading to inconsistent display.
Multi-Action Bar Order
The sorting Action Bar should always be placed on the far right, with the order from left to right being Sort Up, Sort Down. When multiple Action Bars are displayed side by side, different types of buttons may be used as appropriate.

We also include decision rules for selecting the appropriate selection component, whether it is a checkbox, radio button, or dropdown, based on the number of options and the expected user flow. For example, a dropdown should only be used when there are more than seven options; otherwise, a radio selection is recommended for single-choice scenarios.

06. Results

Within a year, we streamlined the design system by cutting component variants 33% (60 → 40) and adding key features like a multi-date picker and form filters. We set UX guidelines, built a Figma library, and aligned workflows with engineering to keep versioning and testing consistent.

By introducing reusable components early, refining patterns, and leveraging AI-powered prototyping, we boosted efficiency, reduced rework, and reached 90%+ completion.

Most importantly, we established a strong rhythm between designers, engineers, and PMs, turning the design system into a shared language and decision-making framework.

SaaS Onboarding Revamp

KKDAY REZIO · 2024–25

Product overview

KKday Rezio is a B2B booking and operations platform for travel businesses, covering product listing, payments, and order management. It integrates with channels such as GetYourGuide, Viator, KKday, and Activity Japan to expand distribution and streamline operations. The platform is trusted by a diverse range of premier global travel brands, including hospitality and leisure experts like Alpina Resorts and Holiday XP, as well as landmark nature destinations such as Izu Panorama Park, Jozankei Farm, and Otaru Canal.

My Role:

As the lead interaction designer, I redesigned the end-to-end product listing experience by restructuring the information architecture, rebuilding the design system, and aligning teams around research findings. The redesign reduced setup time by 38% (45 → 28 min), achieved a SUS score of 80.3, and drove a 30% reduction in support ticket volume.

Timeline

2024–2025

Website

rezio.io ↗

01. Achievement

Product outcomes

1. Reduced setup time by 38% (45 → 28 min), enabling more independent onboarding without support intervention
2. Achieved a SUS score of 80.3, indicating a significantly clearer and more learnable setup flow
3. Reduced support ticket volume by 30% without adding headcount

Team outcomes

1. Compressed a three-month iteration cycle through AI-assisted prototyping, validating hypotheses before development
2. Reduced recurring design-dev handoff friction with a shared component system and version-controlled guidelines

02. Where We Started

Setup Completion Rate
Started
100%
Step 1
82%
Step 2
65%

Step 3
50%
Finished
40%
60 % needed support
45 min avg setup time

Project Background:

Originally built for a narrower user group, the product struggled as operational complexity grew. The "Getting Started" section alone drove 36% of all help article views, signaling deep setup friction. What was framed as a 10-minute setup routinely took 45-50 minutes, and 60% of users could not complete it without support.

Our Goal:

We redesigned the product listing experience to better match users' mental models by restructuring flows, rewriting labels, and surfacing guidance contextually, so users could complete setup with less confusion and less dependence on support.

03. Research and Problem Framing

Preliminary research:

We interviewed users and frontline staff to map how products were actually listed in practice. One charter vehicle operator shared: "I miss the instant replies on LINE. With the new system, responses take much longer."

The pattern was consistent across interviews · system complexity had created a dependency on real-time support that neither users nor the support team could sustain.

Persona:

Research surfaced two distinct user types: Takeshi Tanaka, the business decision-maker optimizing for revenue and channel integration, and Mari Yamaguchi, the daily operator managing orders and reservations. Despite different goals, both hit the same wall · setup complexity that stalled progress and eroded trust in the system.

	Decision Maker Takeshi Tanaka	Daily Operator Mari Yamaguchi
Position	Manager / Business Owner	Operations Assistant / Reservation Agent
Key Focus	Revenue growth, operational efficiency, channel integration	Ease of use, error reduction, quick order access
Key Features	Multi-platform product management, sales reports	Order export, ticket redemption
Needs	Stable integration, enhanced brand image	Easy to learn, time-saving and efficient

User Journey Map:

We mapped emotional engagement across six stages · from first adoption through potential cancellation. Mari's journey surfaced friction concentrated in daily task execution; Takeshi's revealed risk aversion at

strategic decision points. Both paths converged on the same two critical drop-off zones.

😊 Feelings & Thoughts 😞

Mari Yamaguchi
"Bookings pile up and I'm scared of making mistakes."
Mari Yamaguchi
"Finally someone's helping. I hope this fixes the chaos."
Takeshi Tanaka
"System aligned with our processes · promising start."
Mari Yamaguchi
"First tour published. Step-by-step made it stress-free."
Takeshi Tanaka
"Sales clear. I can adjust strategy without guesswork."
Mari Yamaguchi
"Automated emails save me daily. Far fewer mistakes."
Mari Yamaguchi
"Fixing errors wasn't straightforward · always had to call support."
Takeshi Tanaka
"Impact was more limited than expected. Decided not to renew."

Steps

Before using Rezio
Subscribed to Rezio
Uploading Itinerary
Selling Itinerary
Adjusting after Launch
Unsubscribing

Actions

Daily Operator**Mari Yamaguchi**

- Consolidate bookings
- Track schedules
- Send emails manually

Daily Operator**Mari Yamaguchi**

- Onboarding call
- Activate trial

Decision Maker**Takeshi Tanaka**

- Sign contract
- Begin training

Daily Operator**Mari Yamaguchi**

- Publish first tour
- Align team on process

Decision Maker**Takeshi Tanaka**

- View sales reports
- Manage orders

Daily Operator**Mari Yamaguchi**

- Send vouchers
- Monitor capacities

Decision Maker**Takeshi Tanaka**

- Update pricing
- Adjust tours

Daily Operator**Mari Yamaguchi**

- Handle invalid orders
- Contact support

Decision Maker**Takeshi Tanaka**

- Decide not to renew

Pain Points & Opportunities

The journey maps highlighted two critical drop-off zones: initial onboarding and post-launch configuration. Both stages suffer from a fundamental misalignment between the system's architecture and the user's mental model. This gap prevents users from self-correcting when errors occur, leading to:

- Onboarding: Early-stage churn driven by uncertainty over the system's value.
- Post-Launch: Friction and perceived instability when attempting to modify live settings.

😊 Feelings & Thoughts 😞
Steps
Before using Rezio
Subscribed to Rezio
Uploading Itinerary
Selling Itinerary
Adjusting after Launch
Unsubscribing
❤Pain Points

- **Onboarding uncertainty** · Users were unsure whether the system could truly resolve their existing operational chaos.
- **High onboarding cost** · Significant time and internal training were required.
- **Chain reactions after updates** · Complicated update processes frequently triggered downstream errors.
- **Heavy reliance on support** · Workflows for updating pricing or tour details were too complex, making it easy for staff to make mistakes.

✦Ideation

- Build early trust and reduce onboarding friction Example
- Role-based interactive onboarding flows Example
- Example templates, Success Showcases Example
- Guided setup checklist Example
- Simplify update workflows and reduce errors Example
- Preview-before-publish Example

- Auto-validation and error detection Example
- Version history with rollback Example
- Self-serve help panel with step-by-step guidance and FAQs Example
- AI-powered assistant Example

05. Our Solution

New Proposal: Setup That Follows the Way You Think

We decoupled pricing from identity-based time slot bindings · allowing users to assign price rules directly to time periods. The restructured flow follows users' natural working sequence, eliminating the abstract "identity relationship" concept that consistently blocked progress during testing.

01

Package Settings

The entry point for creating a new product listing. Clicking "+ Add Package" triggers a profiling questionnaire before proceeding · replacing the old flow where every field was exposed upfront regardless of product type.

02

Profiling Questionnaire

A short questionnaire captures the itinerary type at the very beginning, so the system only presents fields relevant to that product · hiding everything the user doesn't need. In usability testing, the questionnaire achieved 96% profiling accuracy · enabling users to complete setup end-to-end without backtracking or contacting support, and reducing onboarding training time for the sales team.

03

Item Settings

Configure identity types (e.g. Adult, Child) and specifications for each package. A live preview "Effect Illustration" shows exactly how the booking page will appear to customers · replacing the old flow where merchants had to publish first and then check for errors.

04

Sales Calendar

Set the available sales period and reservation rules · opening time, closing deadline, and excluded dates · all in one view. The live calendar beside the form is a real-time preview: for example, if today is 10/12, the 12th already shows as unavailable because the closing deadline requires 1-day advance booking, and the 18th is marked unavailable because it was added as an excluded date · every rule is visualized the moment it's set. Based on our research, we moved this configuration from the product level down to the package level · so each package can have its own blackout dates and booking window rules. In the old flow, availability was buried across separate tabs with no visual confirmation.

05

Session Settings

Define available days and time slots in one step. Select which days of the week to open, set session start times and durations, and the calendar on the right renders instantly the full weekly timetable · what you configure is exactly what customers will see. The old flow required merchants to pre-create time bindings in a separate global settings area before they could be selected here.

06

Price Settings

Assign prices directly to time slots · with support for multiple price tiers (e.g. Default Price, Weekend Price). The color-coded calendar grid updates in real time, letting merchants verify exactly which price rule applies to each session before publishing · eliminating the old "Bound Rules" model where N×M×T combinations had to be created and verified individually.

Reducing Support Dependency with a Contextual Help Panel

The self-serve help panel surfaces contextual guidance at the exact point of friction · enabling users to resolve issues without contacting support. Estimated impact: ~30% reduction in support ticket volume and faster average resolution time, directly reducing operational cost.

Creating a More Conversational UX Copy System

We replaced isolated field labels with sentence-based inputs · embedding context directly into the form structure. Instead of interpreting a label in isolation, users read a complete sentence and fill in the blanks. Testing showed faster task completion and fewer misconfigurations, contributing directly to the overall SUS score of 80.3.

Reservation Quantity Settings
Manage the sales quantity of the plan and the rules for deducting quantities.
Reservation Quantity Settings*
Ordering 1 adult will deduct 1 from the available quantity.
Ordering 1 child will deduct 1 from the available quantity.
Identity-based ordering restrictions (optional)
This setting will add relevant conditions for identity-based purchases.
1 Children ▾ must be accompanied by 1 Adult ▾

06. Results

The redesign delivered measurable progress against all three goals: setup time dropped from 45 to 28 minutes (–38%), SUS reached 80.3, and support ticket volume dropped by 30% without additional headcount.

AI-assisted prototyping shortened the iteration cycle, while the shared design system reduced recurring handoff friction between design and engineering · improvements that continued beyond the project itself.

With more time, I would involve users earlier in stakeholder alignment so evidence could shape consensus sooner and reduce rework. Even so, one of the strongest outcomes was the team capability built through the process itself.

Bicycle Lane Inspection

LSBG HAMBURG · 2024

Product Overview:

BumpHunter is an AI-assisted service concept for Landesbetrieb Straßen, Brücken und Gewässer (LSBG), designed to turn citizen feedback into structured maintenance insight. By combining Google Maps, Strava activity data, and city weather information, the system helps engineers identify issues earlier, prioritize repairs, and make better-informed infrastructure decisions.

My Role:

As the interaction designer, I scoped two research tracks covering both residents and bicycle lane engineers, and facilitated 20+ cross-cultural interviews. Throughout the project, I translated findings into design principles, built high-fidelity prototypes, and used rolling research to validate functionality and usability. I also aligned a highly international team around interaction patterns that fit Hamburg's local context.

Timeline:

2023 · UnternehmerTUM, Munich

01. Where We Started

Project Background:

Inspection, maintenance, and planning of bicycle lanes require coordination across multiple government departments and inspection agencies. Although citizens can report issues, many still rely on traditional channels like email, resulting in fragmented information that is difficult to act on. Drone-based solutions can help in some situations, but their effectiveness drops significantly during winter.

Our Goal:

Design a lightweight system that turns scattered signals into timely, actionable updates on bicycle lane conditions, so engineers can reduce on-site inspections, coordinate more effectively across departments, and focus repairs where they matter most.

02. Research and Problem Framing

Preliminary Research:

We combined secondary research with primary data collection to understand the challenges and opportunities in bicycle lane planning and inspection.

Secondary research referenced literature on European bicycle lane infrastructure, including planning processes and inspection checkpoints (e.g., snow-related maintenance). Internal system operation documents from LSBG were also reviewed.

Primary research involved interviews with bicycle lane experts, engineers, and cycling enthusiasts, as well as focus groups and on-site observations.

Key interview quotes included:

"The Netherlands succeeded by building a comprehensive network of dedicated cycle paths.", **Professor Chris Brunlett**

"What is currently lacking is the feedback from citizens regarding their riding experiences on bicycle paths.", **LSBG Bike Lane Engineer**

"If my contributions truly help improve my neighborhood, I would be more willing to report.", **Citizen**

Key Insights:

Research showed that engineers lacked detailed information about which road segments mattered most to citizens and how riders actually experienced them. At the same time, citizens wanted their reports to be acknowledged, creating an opportunity for a more proactive inspection process.

Persona:

Our research revealed 2 key roles shaping the bicycle lane planning and inspection process: the Citizen and the Engineer.

	Bicycle Lane / Road Engineer	Resident / Cyclist
Key Focus	Maintenance and inspection efficiency	Reporting road conditions, improving neighborhoods
Key Features	Technical inspection, data analysis	Submitting reports, tracking responses
Needs	Clear citizen feedback on road quality and priority segments	Assurance that feedback is heard and acted on

While their perspectives differ, both groups faced the same structural gaps: limited communication, incomplete data, and no effective channel for exchanging information in ways that could meaningfully shape planning decisions.

Working Across Languages with AI Tools

- **Transcribe fast**, Used Descript to turn German interviews into text.
- **Summarize smart**, Let Hypotenuse pull out key points with prompt-based summaries.
- **Translate quickly**, Ran outputs through online tools for English context.
- **Visualize instantly**, Used QoQo.ai in Figma to map personas & journeys.

03. Our Solution

Service Blueprint:

The service blueprint reframed maintenance as a two-way loop between engineers and residents. Engineers used maps to identify risk, traffic patterns, and likely maintenance priorities; citizens responded with location-specific reports, photos, and riding feedback. Together, these inputs turned scattered observations into a clearer basis for inspection and repair decisions.

The following sections show how the tool helps engineers assess maintenance priorities through maps and request targeted bicycle lane information from citizens when more context is needed.

Two Styles of Information Visualization:

Map View:

Map View enables engineers to quickly pinpoint priority areas for bicycle lane maintenance by visualizing damage severity, report density, and traffic levels. Color-coded markers make it easy to distinguish between extremely dangerous, dangerous, and minor issues, while traffic overlays reveal usage intensity. This combination helps engineers decide not just where intervention is most urgent, but also where repairs will have the greatest impact.

Report View:

Report View transforms citizen feedback into structured, actionable data, presenting an at-a-glance summary of bicycle lane conditions by district. For example, in Bergstedt, the Uncomfortable rate is 60% and the Dangerous rate is 70%, with key issues including low road traction, numerous potholes, and hazardous obstacles like tree branches. The view also shows report counts and traffic demand distribution (High 30.5%, Medium 4.5%, Low 65%), enabling engineers to quickly gauge both usage and risk levels.

Get Road Usage Information from Citizens:

Send Mission:

When engineers need more context to assess a specific area, they can use Send Mission to directly engage local residents. This feature sends targeted requests to citizens in the program, asking them to capture photos, record observations, and provide detailed feedback about the road segment in question. By gathering on-the-ground insights from actual users, engineers can validate existing data, fill in missing details, and make more accurate maintenance or planning decisions.

Citizen-Assisted Reporting of Bicycle Routes:

Capture cycling experience with simple clicks:

Hamburg residents can play an active role in improving local cycling infrastructure by sharing real-world road usage feedback via the Road Usage Reporting Website. Each submission helps engineers better understand on-the-ground conditions and prioritize repairs that matter most to the community.

How it works:

1. Enter the address of the damaged area
2. Adjust the map pin for accuracy
3. Upload photos of the issue
4. Indicate the level of danger and discomfort

With just a few steps, residents can transform their daily riding experiences into actionable insights, directly influencing maintenance priorities and ensuring safer, more comfortable bike lanes.

04. Project Highlights

Team Collaboration & Workflow

We ran Agile sprints with weekly showcases and testing, using story mapping to keep the work grounded. Most collaboration happened in Figma, with Adobe tools supporting production details. I also brought AI into the workflow to accelerate early communication artifacts and keep experiments legible to the wider team.

Testing, Iterating, Failing, Partnering

The most memorable part of this project was learning how differently Munich and Hamburg approached cycling infrastructure from the contexts I already knew. I once suggested posting flyers on utility poles, only to learn Munich had long buried its cables. That moment stayed with me. Working with such a diverse team sharpened my ability to spot blind spots, absorb feedback quickly, and refine ideas collaboratively across cultures.

A41 — All For One

NONE CAPITAL · 2022–23

Product Overview:

All For One is a fashion commerce app that brings together products from Taiwanese and Korean brands in one mobile experience. Users can browse across stores, split checkout by seller, and track orders in a single flow. The design challenge was to turn fragmented catalog data and cross-store logistics into a shopping journey users could actually trust.

My Role:

I led the redesign of three core flows: the product page, shopping cart, and profile page. The work focused on replacing brittle WebView patterns with a more native mobile experience and creating clearer paths to checkout.

Timeframe:

None Capital · 2021–2022

Platform:

Mobile · iOS

01. Where We Started

Before redesigning the interface, I audited the existing product to understand where the experience was breaking down and what the team needed to scale it more coherently.

Understanding Past Design History:

Before I took on this project, the client had hired other designers to work on the interface. Therefore, before initiating the project, I reviewed previous design documents to understand the overall design direction of the product.

Migrating to Figma & Building a Design System:

The previous designer mainly worked with Adobe Illustrator and Avocode. To better support the product team's workflow, I migrated the existing files to Figma and Zeplin, then built a design system to enable smoother collaboration going forward.

Competitive Analysis & UX Information Architecture Audit:

I analyzed core functions (e.g., product pages, checkout flows), studied industry benchmarks, and reorganized the information architecture and component structure.

Logistics Research:

Because split-order logic varies across e-commerce platforms, I reviewed benchmark flows to better understand how other products handled logistics and fulfillment constraints.

02. Our Solution

a. Product Page Redesign

Swipe the prototype to compare the before and after states.

Previous version

In the previous version, we couldn't hide the original shopping cart and purchase buttons from the WebView-based product page. Moreover, store and product information could only be viewed within WebView, which often caused frustration and confusion for users.

Redesign version

In the redesign, improvements in web-scraped data integration allowed designers to display store and product information natively within the app.

b. Main Features of the Redesign Version

Product Information

Since product descriptions could only be retrieved as one full block, I redesigned the section to be collapsed by default. Users expand it only when needed, reducing disruption during browsing or checkout.

Because the length and structure were unpredictable, I added a "Back to top" button at the bottom once expanded. I tested other options (like "Hide"), but usability studies confirmed "Back to top" was the most intuitive and efficient.

Color & Size Picker

The main challenge came from inconsistent scraped data: colors could appear as "grass green" or "green with rhombus pattern," while size labels varied widely. This unpredictability made fixed layouts impossible.

To solve this, I used a Bottom Sheet design, which balanced information capacity and usability on mobile, providing a more stable and flexible user experience.

c. Shopping Cart Design

Two Shopping Carts (Taiwan & Korea)

In the shopping cart design, international items and other products required separate checkouts, so we decided to split the cart into two sections.

This design was mainly to prevent users from being notified after clicking "Checkout" that they couldn't place all items in one order. After user testing, I introduced Tabs, allowing users to easily switch between carts, while showing the current item count directly on each tab.

Checkout Page

For the Item Cards, we set three goals:

- Clearly display prices so users can understand at a glance.
- Highlight the address for easy re-confirmation.
- Allow flexibility for larger product images.

d. User Profile Page Redesign

Previous version

In the original product plan, the User Profile Page reserved space for a future "user post" feature. However, due to strategy changes, this feature was not implemented, leaving the reserved area wasted.

In an e-commerce app, every section represents a potential touchpoint for products, so unused space meant missed opportunities for user engagement.

Redesign version

In the redesigned User Profile Page, I placed My Order and Customer Service at the top for quick access.

Below, I added Recently Viewed, Shopping Cart, and My Picks carousels, increasing the chances for products to re-enter the checkout flow.

Trip Planner

CLAUDE CODE SKILL · 2026

Project Origin:

As a travel enthusiast, I wanted to share my upcoming Korea → Japan trip with friends and decided to do what everyone online was doing — Vibe Code a website. But as the Claude Code trend swept the tech world, and after using Claude Code Skills extensively at work, a new idea took shape: what if I turned my trip-planning method into a structured Skill — so that anyone could become their own travel planner and set off on the journey they truly want?

My Role:

Product Designer, Full-stack Developer, AI Prompt Engineer. Claude Code Skill, MCP, HTML/CSS/JS

Timeframe:

Side Project · 2026.Mar–Present

Open Source:

github.com/ralinzain-star/trip-planner-skill ↗

01. When Design Artifacts Are No Longer the Defining Axis

In the AI era, if the design artifact (Output) is no longer the defining axis of a designer's work, where does a designer's value manifest? In the past, our sense of achievement came from pixel-perfect craft — grid systems, color specs, we were the "final reviewers" of product visuals and interactions. But as generative AI and Vibe Coding sweep in, when a prompt or a set of structured data can instantly produce a beautiful interface, we must honestly confront a shift: "defining problems" and "building frameworks" are becoming a designer's most important skills.

02. Design Philosophy

Dialogue as Interface

Trip planning is inherently fragmented and deeply personal. People rarely want to fill cold forms, they prefer to talk through their needs. I defined conversation as the core interface. Through natural, multi-turn contextual dialogue, the AI simulates the questioning logic of a professional tour guide, progressively refining the plan, letting the interface flow with the user's thinking.

Experience-Driven

The trip webpage is just one form of visual output, the planning process itself is the soul of this product. The essence of travel lies in the experience: what do you want to feel in a foreign place? How in control of your budget do you feel? How do you balance the everyday with unexpected surprises? I placed the design emphasis on the **guided conversation upfront**. This isn't just data collection, it's helping users complete a deep process of self-clarification, reclaiming the spontaneous spirit of travel. Real design happens in the dialogue: clarifying intent, resolving anxiety, and ultimately sending someone on a trip that is truly their own.

03. Key Decisions

What is a Skill?

An Agent Skill is a lightweight, open format for extending AI agents. Each skill is a folder with a `SKILL.md` instruction file — plus optional scripts, templates, and reference materials. The agent reads the instructions on demand, gaining deep, specialized capabilities while staying fast.

On the technical side, I made a core shift: abandoning rigid fill-in templates in favor of designing a structured Markdown as the semantic protocol for the UI. This MD document doesn't just carry travel information — it directly defines the orchestration logic of the UI.

1

Semantic Mapping

I defined a mapping ruleset where different Markdown syntax tags directly trigger specific design system components. Data tables are parsed and rendered as interactive budget dashboards with dynamic filtering. Lists, based on content characteristics, automatically render as checkbox-equipped packing checklists. Geo-links are recognized by the engine and automatically pinned on the underlying interactive map.

Data Table → Budget Dashboard

```
MD Syntax                                    Renders As
─────────────────────────────────────────    ──────────────────────────────

| Item   | Cost  | Status    |                Interactive Budget Dashboard
| Flight | $420  | Confirmed |        →      Filter by city / category
| Hotel  | $850  | Pending   |               Real-time totals + remaining budget
| Dining | $30   |,          |
```

Geo-link → Interactive Map

```
MD Syntax                                    Renders As
─────────────────────────────────────────    ──────────────────────────────

[Tsukiji Market](geo:35.6654,139.7707)     Leaflet Interactive Map
[Sensō-ji](geo:35.7148,139.7967)      →     Auto-pin coordinates
[Shinjuku Gyoen](geo:35.6852,139.7100)     Color-coded by date + route lines
```

2

Giving AI Orchestration Authority

Markdown is the AI's native language. Through MD, I handed the orchestration authority back to the AI, letting it spontaneously design the layout based on each trip's characteristics. For an international flight itinerary, the AI autonomously decides to generate a warning block. For a remote-work-focused trip, it prioritizes a Workspace Guide section. The UI structure is no longer fixed, it grows from the content.

```
Scenario A, International Trip (Korea → Japan)
AI detects border crossing, auto-inserts at page top:

  > ⚠️ Entry Requirements
  > Korea: K-ETA required (apply 72hr ahead)
  > Japan: Visa-free 90 days, but fill out Visit Japan Web
```

```
  > Exchange: 1 TWD ≈ 42 KRW / 5 JPY

──────────────────────────────────────────
Scenario B, Remote Work Trip (Fukuoka Work Mode)
AI detects Nomad Mode, auto-prioritizes:

  ## 🖥️ Workspace Guide, Fukuoka
  | Location           | Wi-Fi  | Outlet | Hours         |
  | Starbucks Tenjin   | 50Mbps | ✅     | 07:00–22:00   |
  | &And Hostel Hakata | 80Mbps | ✅     | 24hr (guest)  |

→ The UI layout isn't a fixed template, it grows from this trip's characteristics
```

04. System Architecture

The system is split into two layers: a conversation orchestration layer (the trip-planner skill that guides the user through a structured multi-phase dialogue with explicit confirmation gates) and a modular skill pipeline (6 specialized sub-skills, each an independent module with its own `SKILL.md`, invoked conditionally by the orchestrator).

Layer 1: Conversation Orchestration

The trip-planner skill acts as the orchestrator. It guides the user through a multi-phase structured dialogue, with each phase ending in an explicit confirmation gate — the system never assumes, it always asks. Phases progress from research loading, through budget and preference gathering, to attraction selection (must-go vs. if-time-permits), then route optimization. Only after every phase is confirmed does the system proceed to generation.

```
 Phase 0  Load Research     ← auto-read travel-collector JSON
Phase 1  Gather Details    ← dates, budget, style, work mode
         ↳ [conditional]   → invoke flight-intelligence
Phase 2  Select Attractions← interactive checkbox batches
         ↳ must-go vs. if-time-permits
Phase 3  Route Optimization← geo-reorder + crowd-aware scheduling
Phase 4  Generate & Deploy ← ui-style → html-generator → deployer
         ──────────────────────────────────────────────────
         Each phase ends with ✓ Confirmation Gate
```

Layer 2: Modular Skill Pipeline

Each skill below is an independent module. The orchestrator invokes them conditionally — flight-intelligence only triggers when flights aren't booked; ui-style only when the user is ready to choose a visual identity. This means each module can evolve independently without affecting the others.

```
 travel-collector → flight-intelligence → trip-planner
                                                ↓
                              ui-style → trip-html-generator → trip-deployer

Each module has its own SKILL.md; the orchestrator invokes conditionally:
```

```
  • flight-intelligence  only when flights aren't booked
  • ui-style             only when the user selects a visual style
```

1

travel-collector

Extracts structured data from any input — URLs, screenshots, blog posts, booking confirmations — and normalizes everything into a unified JSON research file that pre-fills the orchestrator conversation.

```
Input: blog URL / screenshot / booking email
  ↓
Output: busan-research.json
  { "pois": [...], "hotels": [...], "passes": [...] }
```

2

flight-intelligence

Analyzes flight pricing patterns, seasonal trends, and festival calendars. Returns a buy-now-or-wait recommendation with interactive price visualization.

```
TPE → PUS  Mar 30   $180  ← current
TPE → PUS  avg.     $210  ← 6-month average
Verdict: BUY NOW (12% below average, cherry blossom peak)
```

3

trip-planner

The conversational orchestrator (Layer 1). Manages all dialogue phases, maintains cross-round context, and coordinates when to invoke other skills.

```
Round 1: "Solo or group?" → Solo + Work Mode
Round 2: "Budget range?"  → $80-120/day excl. flights
Round 3: "Must-see?"      → Cherry blossom, local food
  ↳ Context persists across all rounds
```

4

ui-style

Provides 30+ design themes (minimalist, editorial, cyberpunk, etc.). The user previews and selects before generation; the chosen style's tokens flow into the HTML generator.

```
After the user selects the "luxury-editorial" style, Style Tokens are injected:
```

```
  --color-primary:   #2C2C2C       --font-heading: 'Playfair Display'
  --color-accent:    #C8A96E       --font-body:    'Source Sans Pro'
  --color-surface:   #FAF7F2       --border-radius: 0px
  --shadow:          0 2px 12px rgba(0,0,0,0.08)
```

```
  → These tokens are injected directly into trip-html-generator
  → Same trip data + different style = completely different visual experience

Other styles: swiss-minimalist | botanical | neo-brutalism
              cyberpunk | newsprint | warm-editorial | ...
```

5

trip-html-generator

Transforms confirmed trip data into a self-contained travel website with interactive maps, calendar views, expense tracking, booking status, and packing checklists. Single bundle, no server required.

```
busan-fukuoka-2026/
├── index.html    ← tabbed UI (Overview/Itinerary/Tickets/Places/Budget)
├── style.css     ← design tokens from ui-style
├── app.js        ← map, calendar, budget tracker, currency switcher
└── data/trip.json
```

6

trip-deployer

One-command deployment to GitHub Pages, Netlify, Vercel, or Cloudflare Pages. Turns the local website into a shareable live URL.

```
Deploy target: GitHub Pages
  → push to ralinzain-star/busan-fukuoka
  → https://ralinzain-star.github.io/busan-fukuoka/
```

05. Live Results

This Skill generated the travel guide I actually used: Busan & Fukuoka 14-Day Trip. Below are the core features of the generated app — each one emerged organically from the Markdown semantic protocol.

1

Splash & Overview Dashboard

A full-screen cover page with a personal travel photo backdrop opens to a live dashboard with four stat cards, a seconds-precision countdown, and today's timeline. Swipe up to enter; the cover flies away with a cubic-bezier animation.

Splash Cover
Custom background · Swipe-up · Today preview
Overview Dashboard
Stat cards · Countdown · Today card · Weather

2

Smart Itinerary Calendar

A week-view calendar with a real-time Now Line red indicator that updates every 30 seconds. Events are color-coded across 8 categories, with embedded restaurant info, booking status icons, and dual KR/JP vs. TW clocks running side by side.

Now Line · Day 5
Live 13:45 indicator on Apr 3
Week 2 (4/6–4/12)
Fukuoka · Work Mode · Itoshima · Return
3

Booking Intelligence & Budget Tracker

Flight cards with route visualization and historical price analysis (green = below avg, red = above). Three-platform comparison (/ Klook / KKday) with lowest-price stars. Budget tracker toggles between estimated and actual modes, with four-currency switching (TWD/KRW/JPY/USD) and city/category bar charts.

Booking & Comparison
Flight · Hotel · Ferry · Activity
Estimated Mode
Total · City ratio · Category bars
Actual Mode
Real expenses grouped by date
4

POI Map & Pre/Post-trip Tools

24 points of interest on an interactive Leaflet map with city and category filter chips, color-matched markers, and solo-traveler tips (e.g., “Sashimi platters start at 2 servings”). A six-category pre-departure checklist persists via localStorage; the post-trip Retrospective shows footprint map, budget vs. actual, and a lessons-learned change log.

Attractions Map
Filter chips · Color markers · Solo tips
Pre-departure Checklist
6 categories · Persistent
Retrospective
Footprint · Budget review · Lessons
5

Time-aware Features & 4-Language i18n

The site adapts to “what time is it now”: a full-screen Today Overlay shifts its checklist based on days-until-departure (>7d / ≤7d / ≤3d / ≤1d). During the trip, today’s timeline dims past events and highlights the current one. All UI text, 8 category names, 14-day weather descriptions, and event titles are translated across zh-TW / English / 한국어 / — 120+ keys, switching takes effect instantly.

Before the Trip
5 days countdown + pre-trip to-dos
During the Trip
Day 5 · Busan→Ferry timeline
After the Trip
Trip Completed · 96 events · 5 cities
06. Reflection

The evolution of this project is a micro-experiment in product design — constantly "returning to the essence." It started from a technology-driven exploration and ultimately arrived at an experience-centered solution.

1. Pivot of Value

Looking back, this project went through three critical shifts in thinking:

1

Starting from "Vibe Coding an App"

Initially attracted by the low-barrier making power of Vibe Coding, my goal was to quickly ship a polished UI app to share my itinerary. The value at this stage was visual presentation.

2

Extending to a "Website-Generating Skill"

With deeper use of Claude Code, I realized systematic capability scales further than any single interface. Packaging planning logic into a Skill that lets anyone one-click deploy a site shifted the value to productivity liberation.

3

Finally Anchoring on "Trip Planning Itself"

Through constant testing, I discovered that visual websites and technical architecture are just means. The essence of trip planning is: how to reduce anxiety, control budgets, and balance the everyday with adventure. I anchored the planning experience and travel essence as the true value core.

2. A Miniature of Product Design Evolution

This "App → Skill → Experience" path mirrors the arc of contemporary product design. It tells us:

- **Interfaces fade, but experiences endure:** In the AI era, UI may be a byproduct of thinking. A designer's job is no longer just defining "where the button goes" but how the system guides users into a fluid experience.
- **Designers are value filters:** Facing infinite AI possibilities, a designer's core competency is defining what truly matters. In this project, I filtered out excess features and kept budget confidence and digital nomad flexibility — what users actually need.

Returning to Design's Origin

This project taught me: the best product lets technology elegantly disappear after completing its task.

The initial Vibe Coding enthusiasm led me into the AI world; my designer's instinct pulled me back beyond the tech framework to the essence of travel. We are entering an era of customized UI — designers define system frameworks, AI handles execution, and users receive the purest, worry-free joy of exploration.

Reflections on Design

ESSAY · 2026

Reflection · 2025

Reflections on Design: From Pixel-Perfect Obsession to the AI Cthulhu

When I was younger, I used to pour my heart and soul into every project for design school, treating every output as a precious treasure. When I first entered the workforce, I held an uncompromising obsession with the completeness and precision of my designs.

But after several years as a digital product designer, I've gradually come to realize that "adaptation" is far more important than being "correct." A process that integrates seamlessly with an organization is better than a set of perfect rules. Rather than chasing pixel-perfect designs, products that are loved, relied upon, used, and bring true value to a company are what's truly worth pursuing.

The emergence of AI has made me realize that the very essence of design is shifting. We are no longer just creators of objects; we are the architects of the fields in which they exist. The key is no longer the artifact itself, but how it intervenes in life, influences human behavior, and creates social value. If a raw, AI-generated suggestion can truly solve a problem or change how people interact, its value far outweighs a polished but ineffective interface. We are moving from delivering outputs to delivering impact. Although job titles have become blurred, our responsibility toward society has only grown heavier.

We are no longer simple builders; we are attempting to tame an uncertain power.

This shift in mindset made Barron Webster's interview resonate deeply with me. His comparison of AI to "Cthulhu" struck a chord. Building products used to be like constructing a house — the logic was clear and visible. Now, working with AI feels like facing an autonomous, unfathomable alien intelligence. We are no longer simple builders; we are attempting to tame an uncertain power. This process of dancing with the unknown is precisely what makes product design today both fascinating and terrifying.

It has also made me realize that clinging to a specific job title is now meaningless. When models can generate interfaces and code at will, the design artifacts we once obsessed over — those perfect Figma components or visual mocks — are rapidly losing their value. In the AI era, these outputs are merely transient guests; true value no longer lies in what you drew, but in what you drove.

Though the world feels chaotic, as a product designer — or as a product builder — I still believe that design can create dialogue, light up people's hearts, pioneer the future, and strive for a better society.

Pixel-perfect

AI

AI

UI Delivering outputs Delivering impact

Barron Webster AI Cthulhu AI

—— Figma —— AI

—— Product Builder ——

" " " "

Pixel-perfect

AI " " "

"

AI

UI " Delivering outputs " " Delivering impact "

Barron Webster AI " Cthulhu " AI

—— Figma —— AI

" " " "

—— Product Builder ——

はデザインスクールの てのプロジェクトに を ぎ てのアウトプットを な のように っていました になった は デザインの と に して のないこだわりを っていました

しかし デジタルプロダクトデザインの で を ごした が しさ よりもはるかに であることに づきました の にシームレスに されるプロセスは なルールのセットよりも れています ピクセルパーフェクト Pixel-perfect なデザインを に い めるよりも に され りにされ に われ に の をもたらすプロダクトこそが たちが に すべきものなのです

AI の により デザインの そのものがシフトしていることに づかされました たちはもはや なる オブジェクトの り ではなく それらが する フィールドの なのです コアとなる は アーティファクトそのものではなく それが々の にどう し の にどう を え をどう み すかへと しました もしAIが した りな が に を したり々の のあり を えたりできるなら その は しく き げられただけの なUIインターフェースをは

るかに　します　たちは　アウトプットの　Delivering outputs　から　インパクトの　Delivering impact　というパラダイムシフトの　にいます　の　は　になりましたが　たちが　に　して　う　はかつてなく　くなっています

この　の　により　Barron Webster のインタビューに　く　しました　AI を　クトゥルフ　Cthulhu　に　えた　の　は　の　を　く　ちました　のプロダクト　は　を　てるようなもので　ロジックは　で　み　げるレンガには　かな　がありました　しかし　AI と　き　うことは　で　り　れない　の　と　するようなものです　たちはもはや　な　ではなく　な　を　なずけようとする　なのです　この　と　るプロセスこそが　のプロダクトデザインを　でありながら　に　ろしくもしている　です

また　の　にしがみつくことがもはや　であることにも　づかされました　LLM　モデル　が　にインターフェースやコードを　できる　たちがかつて　したデザインのアウトプット――あの　な Figma コンポーネントやビジュアルモック――は　に　を　っています　AI　において　これらのアウトプットは　な　に　ぎません　の　はもはや　を　いたか　ではなく　を　したか　にあります

は　に　としていますが　プロダクトデザイナーとして――あるいは　プロダクトビルダー　Product Builder　として――　は　でも　じています　デザインは　を　み　し　の　を　らし　を　り　き　より　い　の　に　けて　できる　を　っていると

Instagram
LinkedIn
WhatsApp: +886 975 573 947
ralinzain@gmail.com